Home Security Monitoring Lab Report

Overview

Over the course of this project I will be building a 3-tier SIEM(Security Information and Event Management) monitoring environment to detect and alert on an SSH brute-force attack in real time. A Wazuh SIEM stack (manager, indexer, dashboard), deployed via Docker on a dedicated Monitor VM(Virtual Machine), collects authentication logs from a victim host and raises alerts, both from Wazuh's default ruleset and from a custom correlation rule I've written specifically for this lab.

Objective/Goals: detect repeated SSH authentication failures from a single source within a short time window, and surface them as a distinct, actionable alert rather than individual noisy log lines.

Result: a working detection pipeline, confirmed end to end by running an actual Hydra brute-force attack and watching the custom rule (ID 100010) fire in the dashboard.

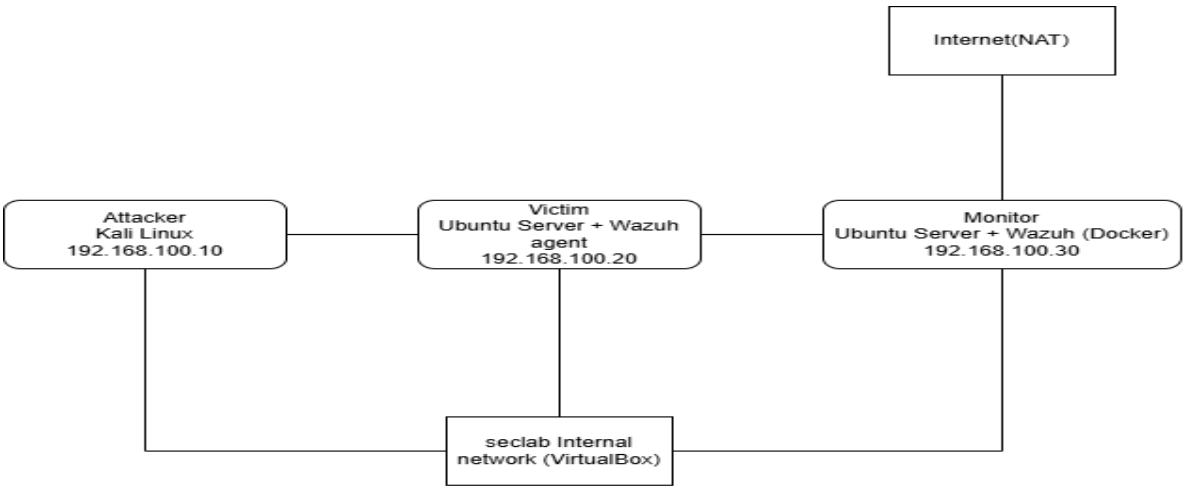
Lab environment

Three VMs on an isolated VirtualBox internal network (called seclab) each also given NAT access for package installs.

VM	Role	OS	IP	RAM	Cores
Attacker	Launches the SSH brute-force attack (Hydra)	Kali Linux	192.168.100.10	2 GB	2
TrueVictim	Target host; generates auth logs; runs Wazuh agent	Ubuntu Server 26.04	192.168.100.20	1.5–2 GB	1
Monitor	Runs the Wazuh stack (manager, indexer, dashboard) via Docker	Ubuntu Server	192.168.100.30	5 GB	4

lab network topology — 3 VMs on seclab

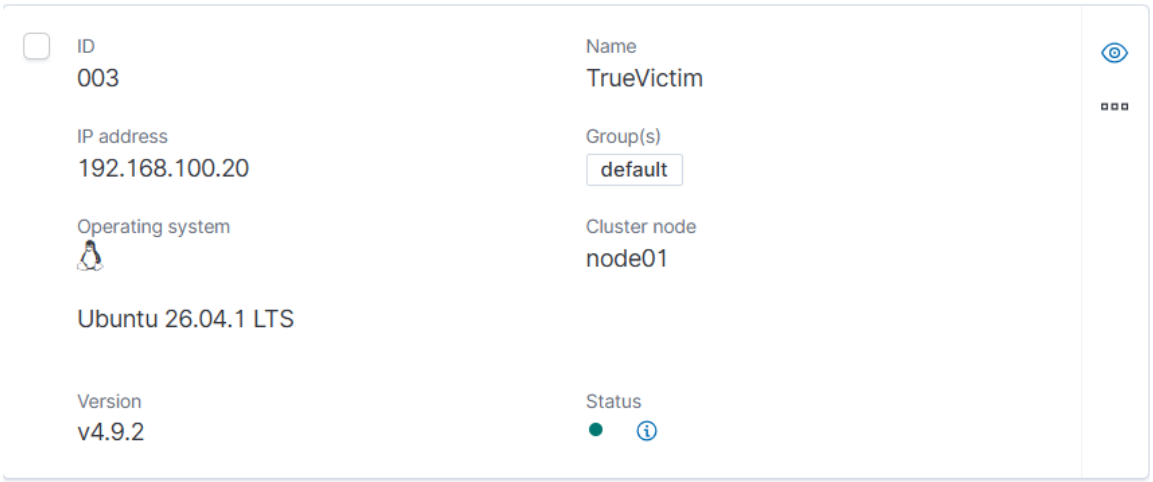
Schematic view of the lab topology: the Attacker, Victim and Monitor all sit on the isolated `seclab` internal network; the Monitor additionally bridges to NAT for Docker image pulls and dashboard access from the host.



The Victim runs a Wazuh agent that forwards ./var/log/auth.log to the Monitor over port 1514. The Monitor's dashboard is reached from the host laptop via port-forwarding(localhost:9443), since direct host-to-internal-network access isn't reliable on restrictive networks.

Build summary

1. **Networking** : static IPs are assigned to all three VMs via netplan (Ubuntu) and NetworkManager (Kali) on a shared internal network.
2. **Wazuh deployment**: the full stack (indexer, manager, dashboard) deployed via docker composer from Wazuh's official wazuh docker repo (v4.9.2), with SSL certs generated for inter-component communication.
3. **Agent enrollment**: a Wazuh agent installed on the Victim via a .deb package, pointed at the Monitor's internal IP, and registered through the dashboard's Agents page.
4. **Verification**: confirmed the agent showed the “Active” status in the dashboard before proceeding to the attack phase.



Attack simulation

An SSH brute-force attack was launched from the Kali VM against the Victim using Hydra with the rockyou.txt wordlist, the standard wordlist of candidate passwords that hydra feeds into each attempt bia the -P flag. Whilst this contains real leaked passwords from the 2009 RockYou breach, this attack isn't designed to succeed, and I ensured that the password used for the truevictim vm wasn't part of this wordlist. Because TrueVictim's actual password is known not to be in the wordlist, the attack was guaranteed not to succeed. This is useful as it proves the detection rule fires on attack behaviour alone, rather than depending on whether the attack succeeds. This allows defenders to be alerted the moment an attack is underway, so they can strengthen defenses or respond, well before an attacker might get lucky or a weak password gets found.

hydra -l vboxuser2 -P /usr/share/wordlists/rockyou.txt -t 1 ssh://192.168.100.20 -t 1 (single-threaded) was used deliberately after an initial run at -t 4 saturated the target sshd's connection limit and caused Hydra to abort with connection errors before any real attempts landed, a useful finding in its own right: a naive high-concurrency brute force can trip a target's own connection throttling rather than its intrusion detection.

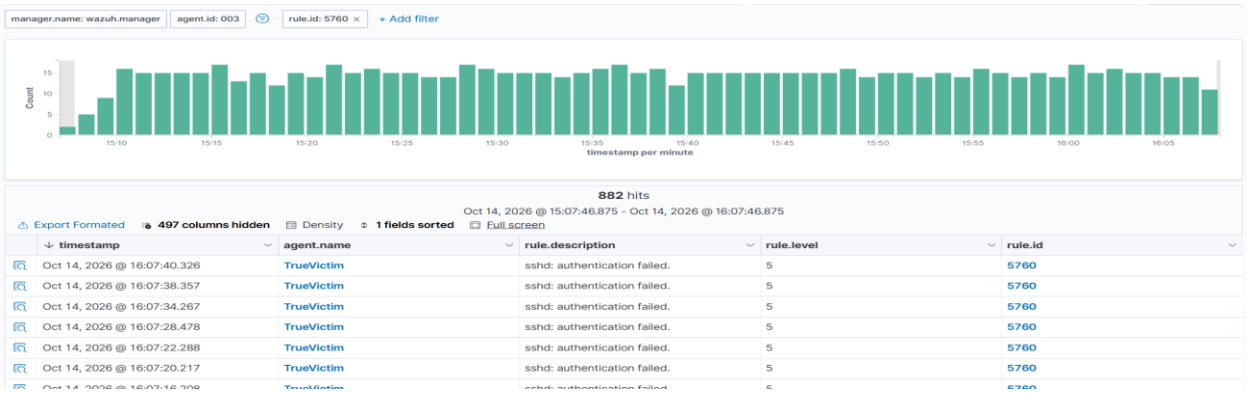
```
┌──(kali㉿kali)-[~]
└─$ nmap -sS -p- 192.168.100.20
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-13 11:25 -0400
mass_dns: warning: Unable to open /etc/resolv.conf. Try using --system-dns or specify valid servers with --d
ns-servers: No such file or directory (2)
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or s
pecify valid servers with --dns-servers
Nmap scan report for 192.168.100.20
Host is up (0.0087s latency).
Not shown: 65534 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
MAC Address: 08:00:27:01:0D:68 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 80.27 seconds
```

Each failed attempt produced a line in the Victim's `/var/log/auth.log`, confirmed live via `tail -f` during the attack: Failed password for vboxuser2 from 192.168.100.10 port 54850 ssh2

Detection and alerting

Once the pipeline was working end to end, each failed login was decoded and matched against Wazuh's built-in rule 5760 ("sshd: authentication failed"). This confirmed the decoder and log-forwarding chain were functioning correctly on a per-event basis.



wazuh-logtest (and its dashboard equivalent, Ruleset Test) was used throughout to validate this, pasting a sample log line in and inspecting the decoder/rule match in isolation, without needing to re-run Hydra for every test.

```
Oct 12 22:00:00 TrueVictim sshd-session[12345]: Failed password for vboxuser2 from 192.168.100.10 port 54321 ssh2
```

▷ Test

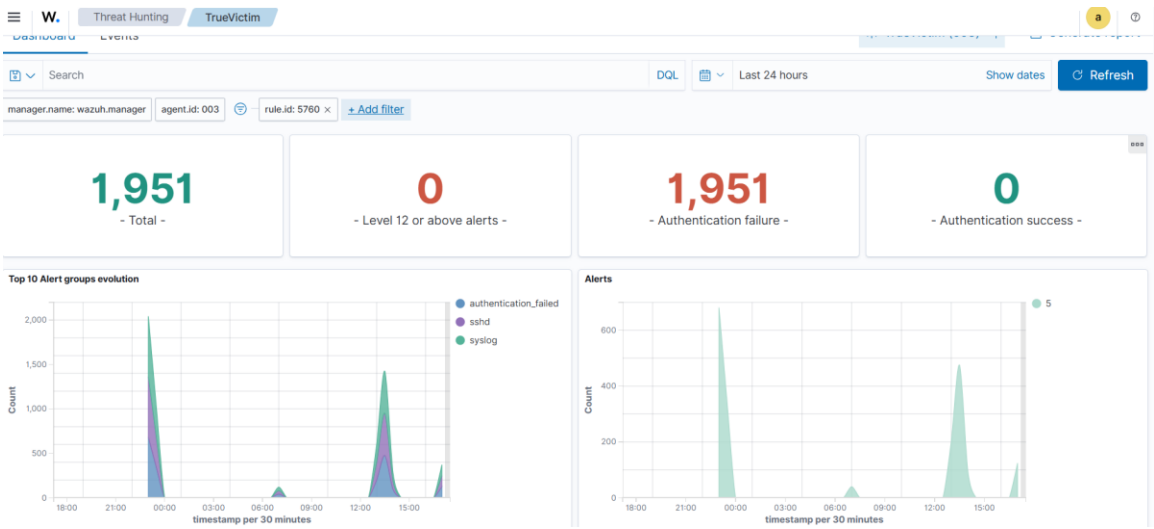
```
**Messages:
        INFO: (7202): Session initialized with token 'd9f243f0'

**Phase 1: Completed pre-decoding.
        full event: '   Oct 12 22:00:00 TrueVictim sshd-session[12345]: Failed password for vboxuser2 from 192.168.100.10 port 54321 ssh2'

**Phase 2: Completed decoding.
        No decoder matched.
```

Something interesting I found was that correlation beat the host's own lockout: in one captured sequence, custom rule 100010 fired at 14:06:00.724, roughly 2 seconds *before* Wazuh's rule 5758 ("Maximum authentication attempts exceeded," reflecting PAM's own max-retry cutoff) appeared at 14:06:02.757. The SIEM's correlation detected the brute-force pattern from the accumulating 5760 events on its own timeline, independent of the host's built-in lockout signal.

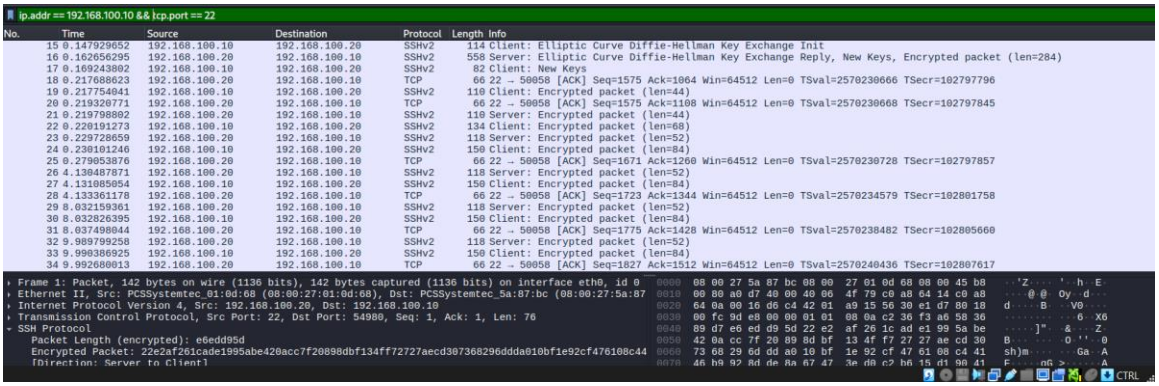
I also notice quite a large alert volume per attempt: a single failed SSH login does not produce one log line, it fans out into several near-simultaneous rule matches from different log sources describing the same event: what was the 5760 (sshd-specific), 5557 (unix_chkpwd), 2501/2502 (generic syslog auth failure), and `5503` (PAM). In a real SOC this redundancy is exactly the kind of noise that gets suppressed or correlated away, rule 100010 effectively does that here, collapsing 6+ of these near-duplicate signals into one meaningful, actionable alert.



Oct 12, 2026 @ 16:35:47.559 - Oct 13, 2026 @ 16:35:47.559

Export Formated | 497 columns hidden | Density | 1 fields sorted | Full screen

	timestamp	agent.name	rule.description	rule.level	rule.id
	Oct 13, 2026 @ 1...	TrueVictim	sshd: authenticat...	5	5760
	Oct 13, 2026 @ 1...	TrueVictim	PAM: User login f...	5	5503
	Oct 13, 2026 @ 1...	TrueVictim	unix_chkpwd: Pa...	5	5557
	Oct 13, 2026 @ 1...	TrueVictim	syslog: User mis...	10	2502
	Oct 13, 2026 @ 1...	TrueVictim	syslog: User auth...	5	2501
	Oct 13, 2026 @ 1...	TrueVictim	Maximum authen...	8	5758
	Oct 13, 2026 @ 1...	TrueVictim	sshd: multiple au...	10	100010
	Oct 13, 2026 @ 1...	TrueVictim	sshd: authenticat...	5	5760
	Oct 13, 2026 @ 1...	TrueVictim	unix_chkpwd: Pa...	5	5557
	Oct 13, 2026 @ 1...	TrueVictim	unix_chkpwd: Pa...	5	5557
	Oct 13, 2026 @ 1...	TrueVictim	sshd: authenticat...	5	5760
	Oct 13, 2026 @ 1...	TrueVictim	unix_chkpwd: Pa...	5	5557
	Oct 13, 2026 @ 1...	TrueVictim	sshd: authenticat...	5	5760
	Oct 13, 2026 @ 1...	TrueVictim	unix_chkpwd: Pa...	5	5557
	Oct 13, 2026 @ 1...	TrueVictim	sshd: authenticat...	5	5760

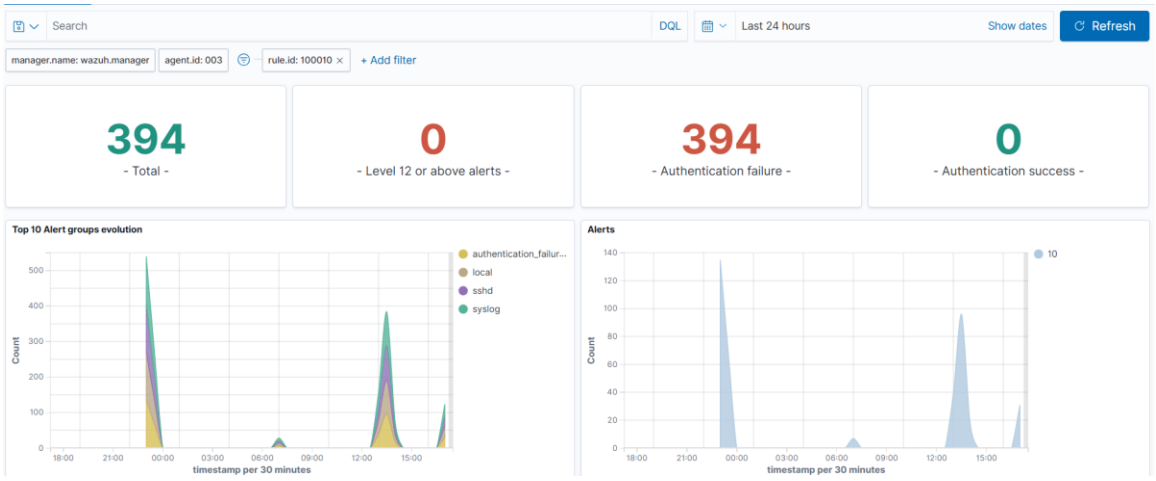


Custom detection rule

Individual failed-login alerts (rule 5760) are noisy and don't by themselves indicate an attack — a user mistyping their password once looks identical to one brute-force attempt. A custom correlation rule was written to raise a distinct, higher-severity alert only when failures cluster in a way that indicates an actual brute-force attempt**:**

```
vboxuser@Monitor:~$ docker exec -it single-node-wazuh.manager-1 cat /var/ossec/etc/rules/local_rules.xml
<group name="local,syslog,sshd,">
  <rule id="100010" level="10" frequency="6" timeframe="120">
    <if_matched_sid>5760</if_matched_sid>
    <same_source_ip />
    <description>sshd: multiple authentication failures from same source - possible brute force attack.</description>
    <mitre>
      <id>T1110.001</id>
    </mitre>
    <group>authentication_failures,</group>
  </rule>
</group>
vboxuser@Monitor:~$ docker exec -it single-node-wazuh.manager-1 cat /var/ossec/etc/decoders/local_decoder.xml
<decoder name="sshd-session">
  <program_name>^sshd-session</program_name>
  <parent>sshd</parent>
</decoder>
```

What it does: if matched sid makes this rule a correlation on top of the base failure rule (5760), rather than matching raw logs directly. frequency="6" and timeframe="120" require 6 matches within a 120-second window before firing, and same_source_ip scopes that count to a single attacking IP, so distributed failures from different sources, or occasional failures spread over time, don't trigger it. The MITRE ATT&CK mapping (T1110.001, Password Guessing) ties the alert to a recognised technique for SOC/reporting purposes. This came out as level 10 as this sits above routine authentication-failure noise (level 5) and signals an event worth an analyst's attention, consistent with Wazuh's severity conventions for credential-access attempts.



Troubleshooting

The custom rule did not fire on the first attempt, despite looking correct on paper. Rather than guessing, each layer of the pipeline was isolated and tested independently using wazuh-logtest (CLI) and Ruleset Test (dashboard), which let a single sample log line be replayed against the manager's decoders and rules without needing to re-run a live attack for every test. Three separate, real faults were found and fixed in sequence:

#	Fault	Symptom	Fix
1	local_rules.xml owned by the host user (1000:1000) instead of the wazuh user	wazuh-logtest warned Could not open file 'etc/rules/local_rules.xml' due to [(13)-(Permission denied)] — the manager could not read the custom rule file at all	chown wazuh:wazuh + chmod 660 on the file, then restarted the manager
2	No decoder matched the log line	Once readable, the rule still didn't fire: Phase 2 of wazuh-logtest showed "No decoder matched". Modern OpenSSH (used by recent Ubuntu) logs under the process name sshd-session instead of sshd; Wazuh's built-in decoder only matches sshd	Added a custom decoder in local_decoder.xml matching ^sshd-session with sshd as its parent, restored correct ownership/permissions, restarted the manager
3	The Victim's Wazuh agent was never configured to monitor /var/log/auth.log	Decoder and rule chain now worked correctly in wazuh-logtest, but live alerts still never appeared on the dashboard; checking ossec.conf confirmed no <localfile> block referenced auth.log — the agent was simply never forwarding it	Added a <localfile> block for /var/log/auth.log with syslog log format, restarted the Wazuh agent

A fourth, more minor issue, the original rule referenced if_matched_sid: 2501, a base SSH rule ID that didn't match the actual log format in use, was also caught via wazuh-logtest, which showed the real base rule firing was 5760, not 2501. The rule was retargeted accordingly (see Custom Detection Rule, above).

This diagnostic approach, isolating decoder, rule, permissions and log-collection as separate, independently testable layers, rather than assuming the first plausible cause, is the methodology take-away of this exercise.

Evidence checklist

- ✓ Network diagram of the 3 VMs and their IPs
- ✓ Screenshot: agent showing Active in Wazuh → Agents
- ✓ Screenshot: Hydra attack running in the Kali terminal
- ✓ Screenshot: rule.id: 5760 alerts in Threat Hunting
- ✓ Screenshot: rule.id: 10010 custom-rule alert in Threat Hunting
- ✓ Final local_rules.xml and local_decoder.xml content (included above)
- ✓ Wireshark capture of the SSH traffic

Conclusion

This lab demonstrated a full SIEM detection workflow: standing up a Dockerized Wazuh stack, enrolling an agent, generating real attack traffic, and writing and debugging a custom correlation rule , including diagnosing and fixing three independent, realistic faults (file permissions, a decoder gap caused by a modern OpenSSH change, and missing log collection config) using Wazuh's built-in testing tools rather than trial-and-error restarts.